

AI Emergency Operations & Resource Orchestration

Bhote Valley Emergency Simulation – evidence-aware disaster response decision support, built on a deterministic fusion, routing and allocation engine.

TEAM LEAD

Om Prajapati

24bsit130@charusat.edu.in

MEMBER

Jeet Pandya

24bca095@charusat.edu.in

MEMBER

Prince Patel

24bsit109@charusat.edu.in

MEMBER

Darshan Mandalaywala

24bca078@charusat.edu.in

Note to judges – team details in submission.yaml still require human confirmation before final submission. All technical content is complete and functional.

THE OPERATIONAL REALITY

Four sources. One valley.
No agreement.

S1 – SATELLITE

Wide, coarse

Sees flood extent across the whole valley, but not who is inside it, and not when.

S2 – FIELD TEAMS

Trusted, sparse

Highest reliability per report, yet only ever covers the few roads a team stood on.

S3 – HOSPITALS

Exact, partial

Precise capacity and surge signal from a single vantage point in the network.

S4 – COMMUNITY CALLS

Fastest, least verified

Arrives before anything else and is the most likely to contradict the rest.

Without a system to fuse this evidence, assess confidence, map accessibility and optimise limited resources against changing conditions, critical decisions are delayed, resources are misrouted, and lives are lost.

Floods, earthquakes and landslides all produce this same shape: simultaneous, uncertain, rapidly changing, partly conflicting.

WHAT WE BUILT

An evidence-aware operational decision engine

Fragmented observations in. Explainable, continuously updated response plans out.

STEP 01	STEP 02	STEP 03	STEP 04	STEP 05
Fuse	Quantify	Reach	Allocate	Replan
Weighted multi-source fusion over every observation about a site or edge.	Confidence attached to each fused value; contradictions raised, not hidden.	Graph accessibility over live road and bridge state; blocked edges excluded.	Constrained assignment of eight resources against ranked, reachable demand.	Every event re-runs the chain and shows the delta it caused.

Deterministic. Explainable. Auditable.

IBM Bob sits on top of the same engine over MCP, so natural-language questions are answered from live backend state.

WHAT WE ARE MOST PROUD OF

Bridge B1 reports both “blocked” and “passable”

REPORT A – FIELD OBSERVATION

B1 — blocked

Carries its own source reliability, confidence and timestamp.

CONFLICT

REPORT B – FIELD OBSERVATION

B1 — passable

Also carries its own reliability, confidence and timestamp.

$w = \text{reliability} \times \text{confidence} \times e^{-\lambda \Delta t}$
freshness decays each observation's weight over time

- 01

Surface it

No silent winner is picked.
- 02

Score it

Reliability × freshness × confidence.
- 03

Verify it

B1 flagged before dispatch.
- 04

Plan around it

Routes assume the worse state.

Uncertainty is a first-class data type, not an afterthought – every decision traces back to specific observations with explainable weights.

FIVE LOAD-BEARING FEATURES

What the engine actually does

-
- 01 **Multi-source evidence fusion with conflict detection**
Reliability × confidence × freshness decay; conflicts flagged before any resource is routed.
 - 02 **Dynamic route & accessibility engine**
NetworkX graph, live road and bridge status; blocked edges dropped, alternates recomputed.
 - 03 **Constrained resource optimizer**
Greedy plus local improvement, capability matching, measured against a nearest-resource baseline.
 - 04 **Continuous replanning**
Each event: fuse, re-score priority, re-route, reallocate – with a visible state change per step.
 - 05 **IBM Bob MCP integration**
14 operational tools reading real backend state; answers cite evidence and algorithmic reasoning.
-

All four engines are deterministic and unit-tested; the language model layer is optional and never decides allocation.

SYNTHETIC BENCHMARK

A mountain valley, wired end to end

SCENARIO COMPOSITION – COUNT

Assets		14
Scripted events		13
Road segments		12
Response resources		8
Bridges		2

14 assets: hospital, 2 health posts, 2 schools, shelter, hydropower, command centre, 5 villages. 8 resources: 3 rescue teams, 2 ambulances, engineering team, supply vehicle, medical unit.

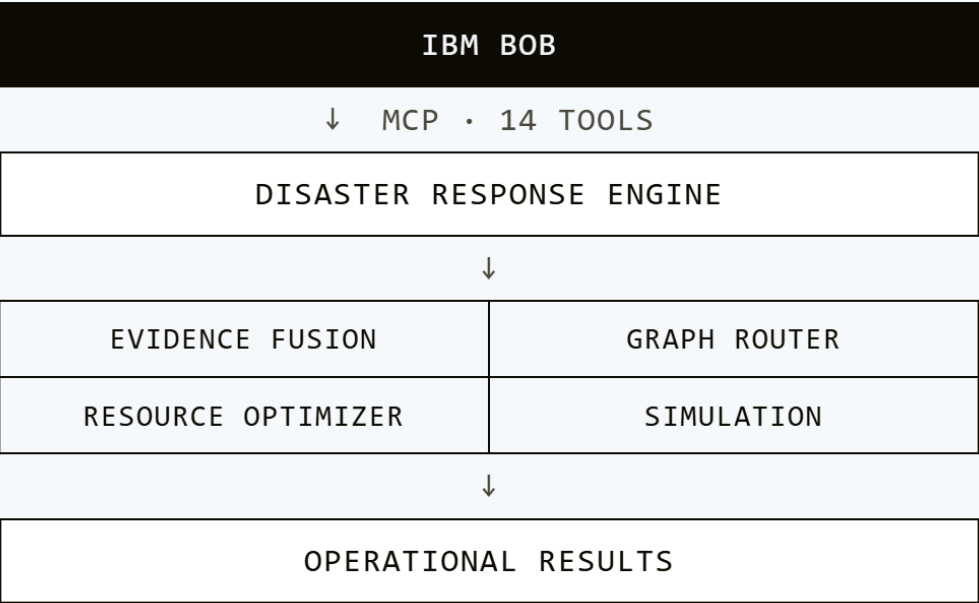
THE 13-EVENT CHAIN – EIGHT PHASES

01	Flood detection
02	Bridge damage
03	Conflicting reports on B1
04	Road blockage
05	Hospital surge
06	Resource failure
07	Route recovery
08	Isolation confirmation

Disclaimer – all entities, coordinates, populations and scenarios are entirely synthetic. This is an anonymised simulation benchmark inspired by mountainous Nepal flood response characteristics, not a reconstruction of any real disaster.

14 MCP TOOLS · NO HARDCODED ANSWERS

Bob asks. The engine answers with state.



"What is the highest priority location?"
→ ranked sites derived from live fused evidence

"Why is Hospital H1 critical?"
→ evidence IDs, confidence, urgency factors

"Can Ambulance A1 reach it?"
→ real route calculation; reports blocked bridges

"What changed after B1 was confirmed blocked?"
→ audit log plus the new priority ranking

"Simulate Road R7 becoming blocked."
→ calls `simulate_event()`; the system replans

Every answer is read from actual backend state through the MCP layer. The full scripted walkthrough lives in `demo/README.md`.

REPRODUCIBLE IN FIVE COMMANDS

Built to be run

LANGUAGES	Python 3.14 · TypeScript
FRAMEWORKS	FastAPI · React · Vite · Pydantic v2
IBM	IBM Bob – MCP server, 14 tools
AI / ML	Deterministic fusion engine; optional watsonx.ai granite-3-8b-instruct
ALGORITHMS	NetworkX · greedy + local improvement · weighted fusion
DATABASE	SQLite – aiosqlite + SQLAlchemy async
OTHER	Leaflet · React-Leaflet · httpx · pytest

REPOSITORY	
src/backend/app/engines/ evidence_fusion · graph_engine priority_engine · resource_optimizer simulation · benchmark	
src/frontend/	React + TypeScript
src/mcp/	14 Bob MCP tools
src/data/	scenario_nepal_inspired
tests/	64 unit + integration
HOW TO RUN	
make install	
make dev-backend	:8000 · /docs
make dev-frontend	:5173
make test	64 tests
make benchmark	metrics

Clone github.com/itsokayyybro/bob-ai-hackathon-shutdown – full instructions in docs/setup-guide.md.

AFTER THE FULL 13-EVENT SIMULATION

Measured against a nearest-resource baseline

METRIC	AI SYSTEM	BASELINE
Task coverage	From actual benchmark run	From actual benchmark run
Total travel time	Optimised – greedy + swap	Nearest-resource greedy
Conflict detection	Active	None
Blocked-route violations	0 – evidence-aware	Possible violations

TEST SUITE

64

pytest unit and integration tests covering fusion weighting, graph routing under blocked edges, priority scoring, assignment constraints and the simulation event loop.

Every metric is produced by make benchmark from actual simulation state and never hardcoded – which is why this slide names the measured metrics rather than printing numbers we did not run here.

KNOWN LIMITATIONS

Where it stops — and why it still matters

- 01 No real-time external ingestion – simulation only, no live sensor feeds.
- 02 Map tiles need internet for the CartoDB dark theme; falls back gracefully offline.
- 03 Optimisation is greedy + local improvement, not full ILP – sufficient at hackathon scale.
- 04 watsonx.ai is optional; the system is fully functional on the deterministic provider.
- 05 Team details in submission.yaml require manual completion.

When two field reports contradict each other, the system refuses to guess. It shows the conflict, weighs the evidence, asks for verification, and plans for the worse case in the meantime.

Demo video – [demo/demo-video-link.txt](#)

Live demo – [demo/live-demo-url.txt](#)

Screenshots – [demo/screenshots/](#) · Slides – [presentation/](#)